

CodeJudge — Final Project Documentation

Goal: Build a small, polished, fully deployed coding-assignment platform in **4–5 days**.

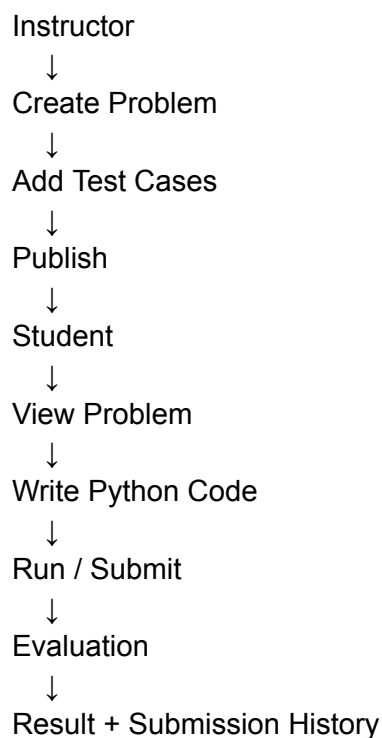
Principle: No unnecessary technologies or features. Every feature must be understandable and explainable in an interview.

1. Project Overview

CodeJudge is a lightweight coding-assignment platform with two roles:

- **Student** — solves programming problems and submits Python solutions.
- **Instructor** — creates problems, manages test cases, and views submissions.

Main flow



2. Tech Stack

Layer	Technology
Frontend	Angular + TypeScript
UI	Angular Material
Code Editor	Monaco Editor
Backend	Python + Flask
API	REST API
ORM	Flask-SQLAlchemy
Authentication	JWT
Database	PostgreSQL
Code Execution	Docker
Testing	Pytest
Version Control	Git + GitHub
Frontend Deployment	Vercel
Backend Deployment	Render
Database Hosting	Neon PostgreSQL

Do NOT use

- Kubernetes
 - Redis
 - Celery
 - Pub/Sub
 - BigQuery
 - Microservices
 - Multiple programming languages
 - Complex AI features
-

3. Roles

Student

Actions

- Register
- Login/logout
- View problems
- Search/filter problems
- View problem details
- Write Python code
- Run code
- Submit code
- View result
- View own submission history
- View own progress
- View leaderboard

Cannot

- Create/edit/delete problems
 - Manage test cases
 - View hidden test cases
 - View other students' private submissions
-

Instructor

Actions

- Login/logout
- View dashboard
- Create problem
- Edit problem
- Delete problem
- Publish/unpublish problem
- Add/edit/delete test cases
- View submissions
- View basic problem statistics

Cannot

- Modify student submissions/results
-

4. Authentication & Authorization

Use **JWT authentication**.

Login flow

User



Email + Password



Flask



Verify password



Generate JWT



Angular

JWT contains:

user_id

role

Roles:

student

instructor

Use Angular route guards for frontend navigation.

Important: Flask must also check the user's role. Frontend guards alone are not security.

5. Student Features

Student Dashboard

Show:

Problems Solved

Total Submissions

Success Rate

And recent submissions.

Problems

Student can:

- View published problems
- Search by name
- Filter by difficulty

Each problem shows:

Title

Difficulty

Solved / Not Solved

Problem Details

Display:

Title

Description

Examples

Constraints

Difficulty

Then display the code editor.

6. Code Editor

Use **Monaco Editor**.

Support only:

Python

Buttons:

RUN

SUBMIT

RUN

Runs the code against **public/sample test cases**.

It does not create a scored submission.

SUBMIT

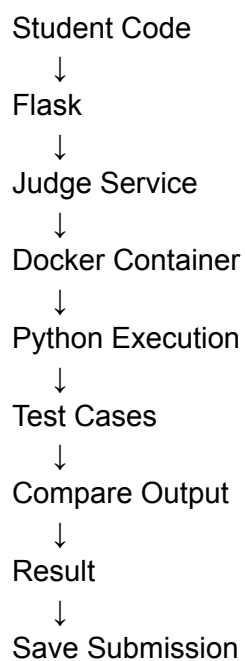
Runs the code against:

- Public tests
- Hidden tests

Then creates a submission.

7. Code Evaluation

Flow



Possible results:

ACCEPTED
WRONG_ANSWER
TIME_LIMIT_EXCEEDED
MEMORY_LIMIT_EXCEEDED
RUNTIME_ERROR

Display:

Status
Score
Passed Tests
Total Tests
Runtime
Memory

Example:

✓ ACCEPTED

Score: 100/100

Tests: 5/5

Runtime: 0.12s

Memory: 12 MB

Students must **never receive hidden test-case input/output**.

8. Docker

Docker is used only for the code execution component.

Basic requirements

The judge should have:

- Execution timeout
- CPU limit
- Memory limit
- Temporary execution environment
- No unnecessary network access
- Container cleanup

Only support **Python**.

Do not build a production-grade multi-language judge.

For the project documentation, describe Docker as an **MVP isolation mechanism**, not as complete security for arbitrary untrusted code.

9. Submission History

Students can view their own submissions.

Display:

Problem

Language

Status

Score
Runtime
Submitted At

Clicking a submission shows:

Source Code
Status
Score
Tests Passed
Runtime
Memory

10. Instructor Dashboard

Display:

Total Problems
Total Submissions
Total Students
Acceptance Rate

Also show basic problem statistics:

Problem
Attempts
Accepted
Acceptance Rate

No advanced analytics.

11. Problem Management

Instructor can:

Create
Edit
Delete
Publish
Unpublish

Problem fields:

Title
Description
Difficulty
Maximum Score
Time Limit
Memory Limit
Status

Difficulty:

Easy
Medium
Hard

Status:

Draft
Published

Only published problems are visible to students.

12. Test Cases

Each problem can have multiple test cases.

Fields:

Input
Expected Output
Hidden/Public

Instructor can:

Add
Edit
Delete

Students can see only public test cases.

13. Leaderboard

Simple leaderboard based on total score.

Rank
Student
Problems Solved
Total Score

No complicated ranking system.

14. Database

Use only **four tables**.

users

id
name
email
password_hash
role
created_at

problems

id
title
description
difficulty
max_score
time_limit
memory_limit
created_by
status
created_at
updated_at

test_cases

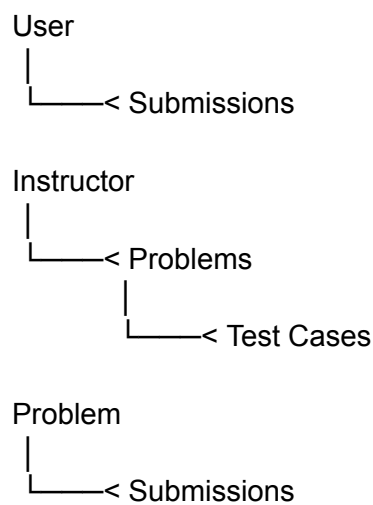
id
problem_id
input_data
expected_output
is_hidden
created_at

submissions

id

user_id
problem_id
language
source_code
status
score
runtime
memory
submitted_at

15. Relationships



16. Backend API

Authentication

POST /api/auth/register
POST /api/auth/login
GET /api/auth/me

Problems

GET /api/problems
GET /api/problems/<id>

POST /api/problems
PUT /api/problems/<id>
DELETE /api/problems/<id>

POST, PUT, and DELETE are instructor-only.

Test Cases

POST /api/problems/<id>/testcases

PUT /api/testcases/<id>

DELETE /api/testcases/<id>

Instructor-only.

Submissions

POST /api/submissions/run

POST /api/submissions

GET /api/submissions/me

GET /api/submissions/<id>

Dashboard

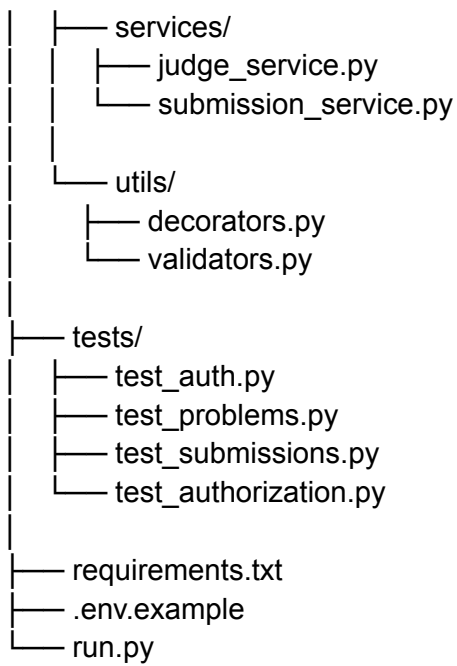
GET /api/analytics/student

GET /api/analytics/instructor

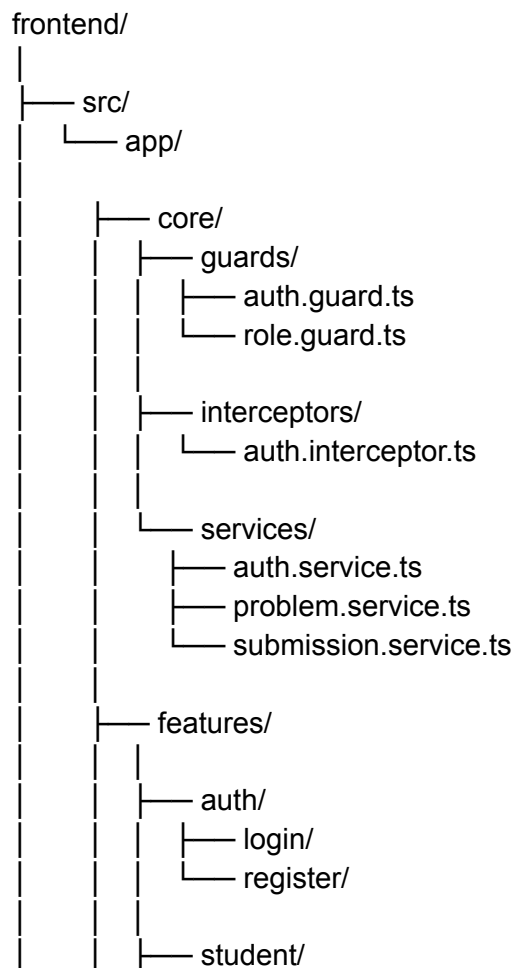
GET /api/leaderboard

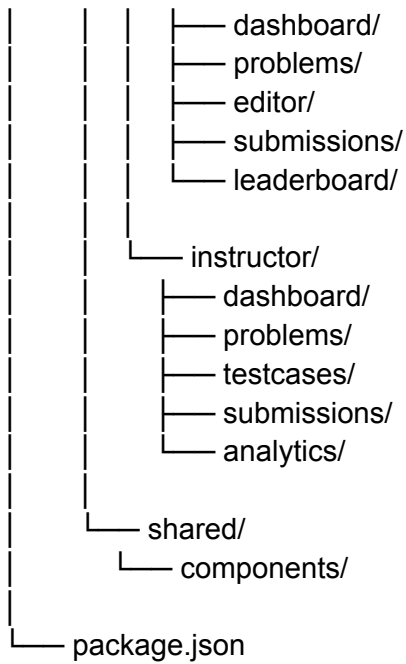
17. Backend Folder Structure

```
backend/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── extensions.py
│   ├── models/
│   │   ├── user.py
│   │   ├── problem.py
│   │   ├── testcase.py
│   │   └── submission.py
│   ├── routes/
│   │   ├── auth.py
│   │   ├── problems.py
│   │   ├── testcases.py
│   │   ├── submissions.py
│   │   └── analytics.py
```

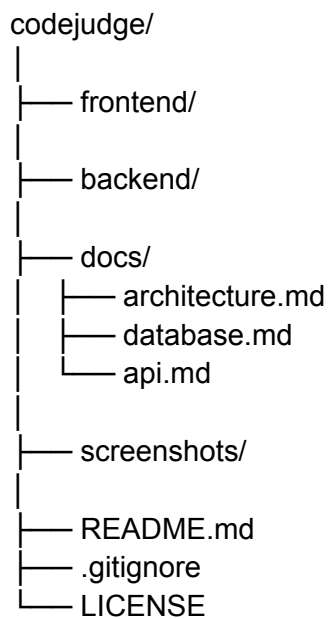


18. Angular Folder Structure





19. Complete Repository

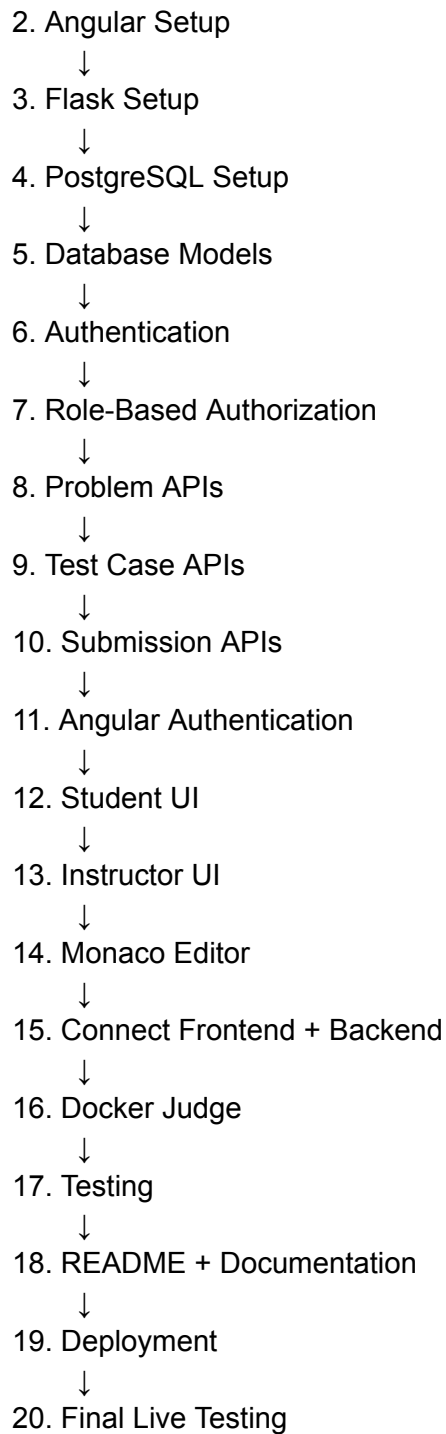


20. Development Order

Follow this order exactly.

1. GitHub Repository





21. 5-Day Plan

Day 1 — Setup + Backend

- GitHub repository
- Angular setup

- Flask setup
- PostgreSQL
- Database models
- Register/login
- JWT
- Basic RBAC

End of Day 1

Authentication should work.

Day 2 — Backend

Build:

- Problem APIs
- Test-case APIs
- Submission APIs
- Authorization
- Basic analytics APIs

Test APIs with Postman.

End of Day 2

Backend should work independently.

Day 3 — Angular

Build:

- Login/register
 - Student dashboard
 - Problems page
 - Problem details
 - Monaco editor
 - Instructor dashboard
 - Problem management
-

Day 4 — Integration + Judge

Connect:

Angular

↓

Flask

↓

PostgreSQL

Then implement:

Run

Submit

Evaluation

Submission History

Add Docker judge.

Day 5 — Testing + Deployment

Testing

Manual testing:

Register

Login

Student permissions

Instructor permissions

Create problem

Add test cases

Run code

Submit code

Accepted

Wrong Answer

Submission history

Pytest

Add only a few important automated tests:

```
test_register()
```

```
test_login()
```

```
test_student_cannot_create_problem()
```

```
test_instructor_can_create_problem()
```

```
test_submit_solution()
```

```
test_get_submission_history()
```

Then:

GitHub cleanup



README



Deployment



Live testing

22. Deployment

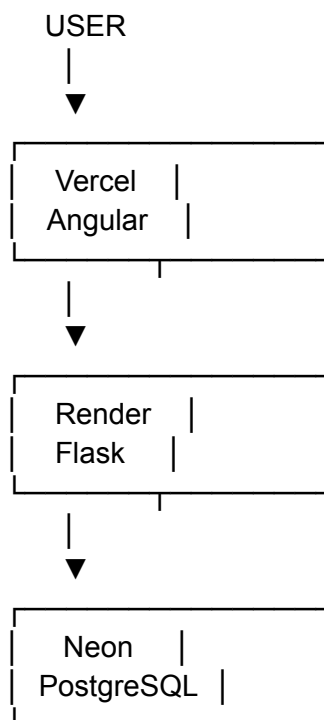
Use:

Angular → Vercel

Flask → Render

PostgreSQL → Neon

Architecture:



The Docker judge can remain a local component if the chosen free backend hosting setup cannot conveniently run it. **Do not let Docker deployment jeopardize the live MVP.**

23. Environment Variables

Backend

DATABASE_URL=
JWT_SECRET_KEY=

Frontend

API_URL=

Never commit:

.env

Commit:

.env.example

24. GitHub Workflow

Use meaningful commits:

initial project setup

feat: add flask api structure
feat: implement authentication
feat: add role based authorization
feat: add problem management
feat: add test case management
feat: add code editor
feat: implement submissions
feat: add docker judge
feat: add student dashboard
feat: add instructor dashboard
test: add backend tests
docs: add project documentation

25. README

The GitHub README should contain:

CodeJudge

Live Demo

Features

Tech Stack

Architecture

Screenshots

Project Structure

Local Setup

Environment Variables

API Overview

Security

Deployment

Future Improvements

Add **2–4 screenshots**:

1. Student dashboard
2. Problem + code editor
3. Submission result
4. Instructor dashboard

26. Security

Implement:

- Password hashing
- JWT authentication
- Backend role authorization
- Input validation
- CORS configuration
- Environment variables for secrets
- Docker-based code execution

Do not expose:

- Passwords
 - JWT secret
 - Database credentials
 - Hidden test cases
-

27. What You Will Demonstrate

This project should demonstrate that you understand:

Frontend development



REST APIs



Authentication



Role-based authorization



Database design



Code execution



Docker



Testing



Git/GitHub



Deployment

This is enough for a **4–5 day portfolio project**.

28. Final MVP Boundary

MUST BUILD

- ✓ Student/Instructor authentication
- ✓ RBAC
- ✓ Problem CRUD
- ✓ Test case management
- ✓ Python code editor
- ✓ Run code
- ✓ Submit code

- ✓ Evaluation result
- ✓ Submission history
- ✓ Student dashboard
- ✓ Instructor dashboard
- ✓ PostgreSQL
- ✓ Docker judge
- ✓ Basic Pytest tests
- ✓ GitHub repository
- ✓ Live deployment

Priority: working end-to-end application → deployment → clean GitHub → testing → UI polish → optional extras.